

Lab Procedure for Simulink

Sensor Fusion

Setup

1. It is recommended that you review [Lab 2 – Application Guide](#) before starting this lab.
2. Launch Quanser Interactive Labs, scroll to the "QBot Platform" menu item and then select the "Warehouse" world.
3. Open the Simulink Model [scan_match.slx](#). This lab builds on **Lab 1: Localization** in Skills Progression 2. Ensure that the relevant sections of the model are completed with your code from Lab 1:
 - a. Open the **LiDAR & Localization** subsystem, then open the **Subsample and Correct LiDAR** subsystem. Open and complete the function block **correct_lidar**.
 - b. Return to the **LiDAR & Localization** subsystem and complete the connections between the subsystems.
4. Open a **second** MATLAB window. Open [sensor_fusion.slx](#). Loading the Simulink model also triggers the setup script, qlabs_setup.m, for the Quanser Interactive Labs virtual environment. Verify that the script ran successfully by checking that the QBot and other elements have been spawned within the virtual environment in Quanser Interactive Labs, as shown in Figure 1. Open the camera menu by clicking the  icon and set the Lock camera location to the "On" position.

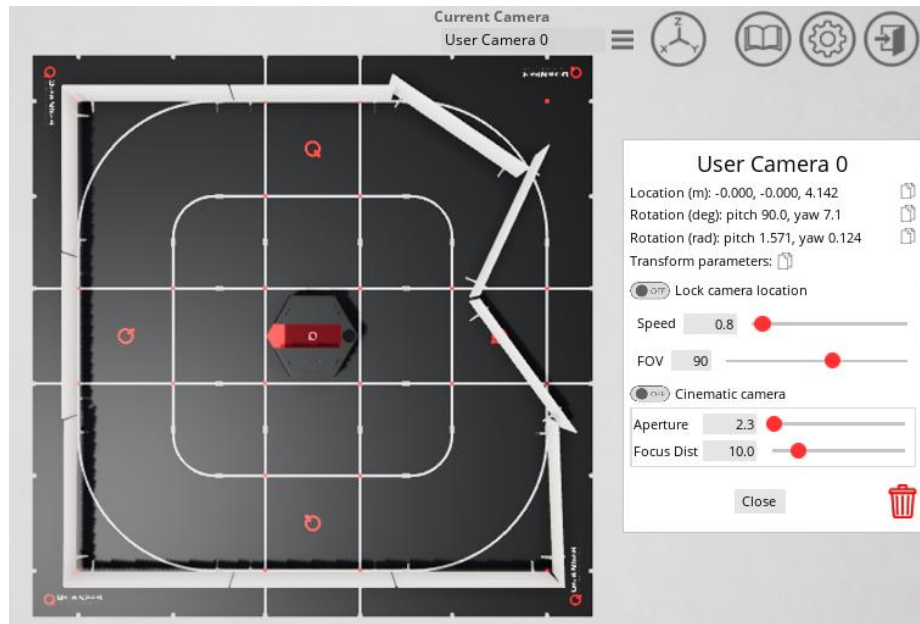


Figure 1. Successful setup of the Quanser Interactive Lab Workspace

Estimating Forward Speed

1. In this lab you will explore sensor fusion techniques to localize the QBot Platform. You should have 2 separate MATLAB instances open: one to run [scan_match.slx](#) and another to run [sensor_fusion.slx](#). The model [scan_match.slx](#) (Figure 2) estimates the position from LiDAR data using Scan Match and streams the estimates to the other model [sensor_fusion.slx](#) (Figure 3). The model [sensor_fusion.slx](#) will include your code to fuse the data from other sensors on the QBot and the received estimates from [scan_match.slx](#).

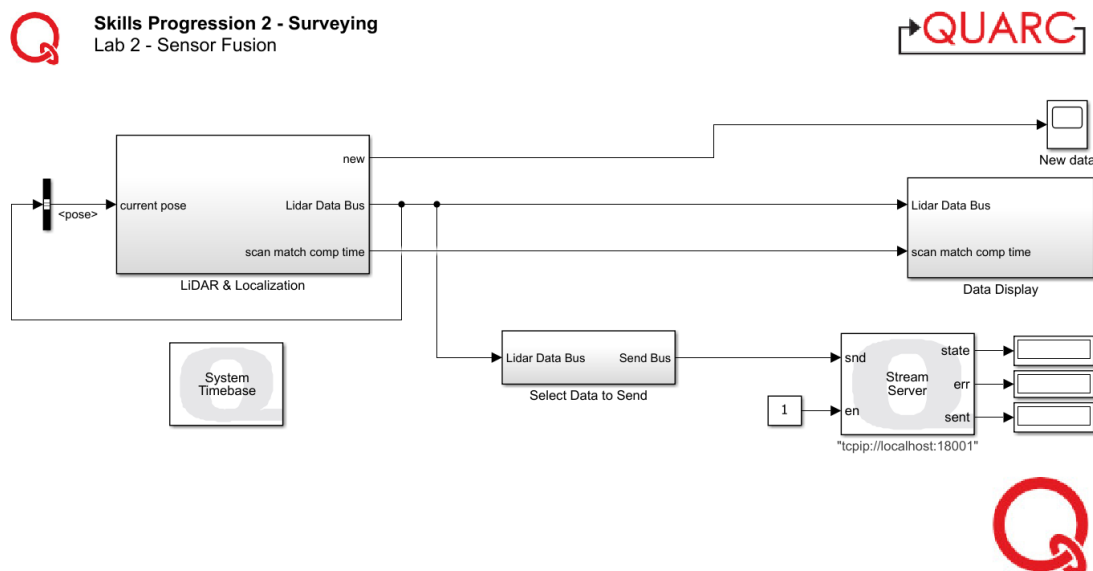


Figure 2: Simulink model in [scan_match.slx](#)

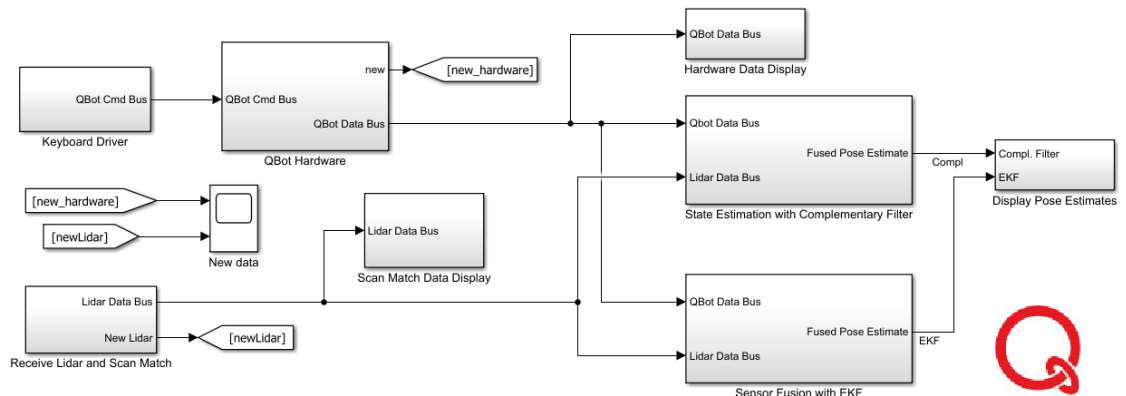


Figure 3: Simulink model in [sensor_fusion.slx](#)

Running the simulation: This lab requires both Simulink models to be running simultaneously. To run a model, click the Run button under the Simulation tab. When instructed to run the models, first run [scan_match.slx](#), and then run [sensor_fusion.slx](#). When stopping the code, ensure that you stop both models using the Stop button under the Simulation tab.

2. In this section, you will construct a complementary filter to fuse two signals together to estimate the body forward speed of the QBot. First, you will explore how we can use the accelerometer signal to estimate the forward speed.

To drive the QBot using your keyboard, press and hold the space bar to arm the robot, and keep the space bar pressed as you drive the QBot using the movement keys. To stop the QBot, disarm it by releasing the space bar. The movement keys are as follows:

- Press the "A" and "D" keys to turn the QBot body
- Press the "I" and "K" keys to drive the QBot forward and back
- Optionally, press the "7" key to drive the QBot in a circle by applying a constant forward and turning speed command.

Run the models by clicking the play button under the Simulation tab and drive the QBot forward while observing the Acceleration scope. Which accelerometer axis corresponds to the forward movement direction? How do we estimate velocity from this signal? What is the expected signal value when the QBot is idle?

- Stop the models. Complete the connection shown in Figure 4 to route the forward acceleration to an integrator block, which is connected to the **accelFwdSpd** block that routes the signal to the Forward Speed Estimate scope.

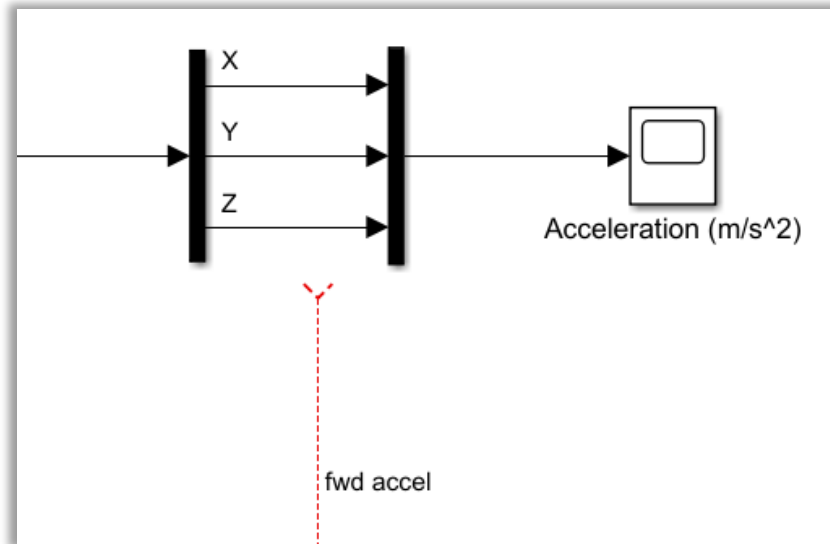


Figure 4: Connecting forward acceleration

4. Run the models. Without driving the QBot around, let the models run for at least 1 minute and observe the Forward Speed Estimate scope. Then, drive the QBot forward and backward using your keyboard keys. Release the spacebar to disarm the QBot and let it come to a stop. Is the forward speed estimated accurately when the QBot is idle and when it is moving? What do you observe about the estimated forward speed when the QBot is stationary? Take a screenshot to show your observations. Stop the models.
5. Next, you will use odometry to estimate the forward speed as done in Lab 1: Forward Kinematics from Skills Progression 1. Connect the Wheel Speeds signal to the **Demux** block that is connected to the **forKin** function block, which calculates the forward and turn speeds from the individual left and right wheel speeds. Connect its output, **vForward**, to the Forward Speed Estimate scope.
6. Repeat Step 4 and observe both signals on the scope. What do you observe about the odometry-based forward speed estimate? How does this estimate compare to the accelerometer-based estimate? Take a screenshot of the scope to show your observations.
7. Now you will fuse the estimates from the individual sensors by correcting for the observed error in the accelerometer-based estimate using a proportional controller with the odometry-based estimate as the reference. In the Complementary Filter section, complete the connections shown in Figure 5: connect the output of the integrator to the + input of the first sum block, then apply a gain of K_p to this output before it is subtracted from the original accelerometer measurement. Set the value of K_p to 0.1.

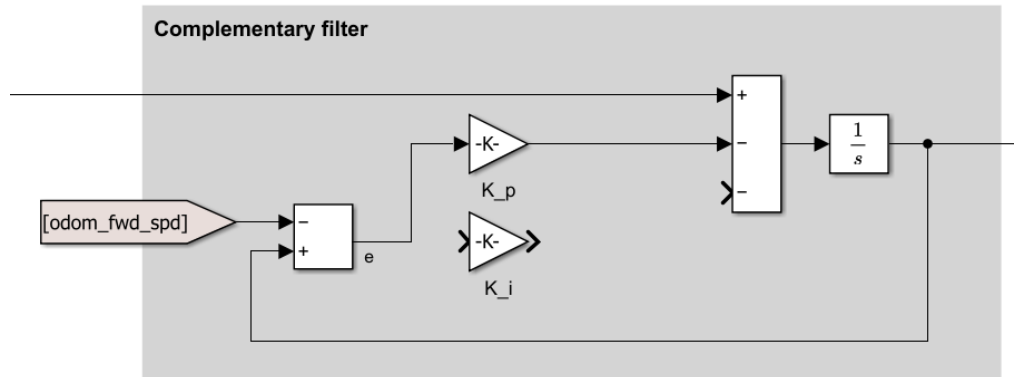


Figure 5: Proportional controller implementation

8. Run the models. In the Forward Speed Estimate scope, enable the **fused forward speed estimate** signal by clicking the signal name on the legend, shown in Figure 6 below. While observing the Forward Speed Estimate scope, let the QBot idle for 30 seconds and then drive the QBot around. Compare the fused estimate to the estimates from the individual sensors. Stop the models.

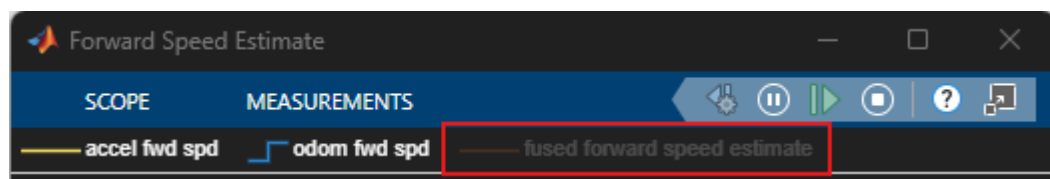


Figure 6: Enable the fused forward speed estimate from the scope legend

9. Run the models. Observe the fused estimate on the scope as you set K_p to different values, including 0.5, 1, 5, and 10. How does increasing K_p affect the fused estimate?
10. Complete the implementation of the complementary filter by adding an integrator to the feedback controller as shown in Figure 7: Connect the output of the calculate error block to the gain K_i . Place an integrator block between the output of the K_i gain and the - input of the sum block.

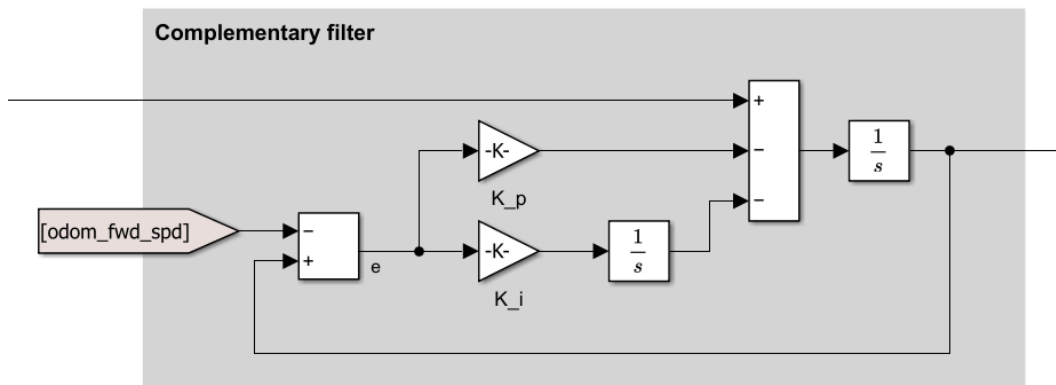


Figure 7: Integral controller implementation

11. Set the value of K_p to 1 and set the value of K_i to 0.01. Run the models and observe the signals on the scope as you increase and decrease K_i . How does the value of K_i affect the fused estimate?
12. Tune the values of K_p and K_i until the fused forward speed accurately tracks the correction signal with minimal noise. A sample tuned signal is shown in Figure 8.



Figure 8: Fused forward speed estimate with tuned K_p and K_i gains

13. Complete the implementations of the other two complementary filters. Start with the Turn Speed Complementary Filter. The rate input to the subsystem is the rate of the turn speed measured by the gyroscope. The correction signal is the body turn speed calculated from the measured wheel speeds. View the individual and fused estimates using the Turn Speed Estimate scope.
14. You now have fused estimates of the QBot's forward speed and turn speed. These are converted to speeds along the X and Y directions and combined into a 3-element vector signal [x speed (m/s), y speed (m/s), omega (angular velocity) (rad/s)]. The Pose Complementary Filter uses this as the rate input and estimates the current position of the QBot using the position estimate from scan match as the correction input, which is a 3-element vector signal [x position (m), y position (m), theta].
15. Tune the K_p and K_i gains for the Pose Complementary filter. Run your code and take a screenshot of the final pose estimate as the QBot idles and as it is driving in a circle.

Comparison to Kalman Filter

16. Now, we will test an Extended Kalman Filter (EKF) for pose estimation using the same sensors and compare this sensor fusion technique to the complementary filter. In the main model [sensor_fusion.slx](#), right-click and uncomment the **Sensor Fusion with EKF** subsystem, which contains a completed implementation of an EKF. The EKF fuses new measurements into an estimate using an optimal weight that minimizes the estimate's uncertainty. Unlike the complementary filter, the EKF does not require manual gain tuning, since the weights are calculated using the motion model and measurement uncertainties.
17. Return to the main model [sensor_fusion.slx](#), open the Display Pose Estimates subsystem, and then open the Pose Estimates scope. Run the code. Drive the QBot in a circle while observing the Pose Estimates Scope. How do the estimates from the EKF compare to the estimates from the complementary filter?
18. Stop the Simulink models. Ensure that you save a copy of your completed files for review later. Close Quanser Interactive Labs.